Grama Jyothi Rural Electricity Co-operative
TASK 4

TASK 4 – Smart-Meter Design Challenge (K5)

Decision: MODIFY the Original Design Retain integer-based monetary arithmetic, validation, centralized tariff data, and clear separation between input, tariff calculation, and reporting. Modify the input model to handle timestamped watt-hour readings every 15 minutes and use a tariff table loaded from a configuration file. A full redesign is not required for the basic calculation, but data ingestion and storage need to be redesigned for scale. Smart-Meter Requirements There are 35,040 fifteen-minute intervals per house per year (365 × 24 × 4). Use a sufficiently wide unsigned integer type for cumulative watt-hours and validate counter rollover based on the actual meter specification. Compute interval energy from consecutive readings; convert watt-hours to billing units using a defined, consistent rule. Apply a 20% surcharge to energy used from 6 PM through 10 PM. Define timezone, timestamp boundaries, missing intervals, and daylight-saving policy if applicable. Load time-of-day rates from a configuration file at startup, validate it, and log which tariff version was used for each billing period.

Grama Jyothi C Programming Assignment Guide Page 5 For m interval records, a chronological streaming calculation can take O(m) time and O(1) working memory beyond tariff data. Storing all intervals takes O(m) memory. Illustrative C Design for One Interval

```
#include <stdint.h>
#include <stdbool.h>
/* Illustrative helper: energy is in Wh; the rate is paise per kWh.
   Real code must validate timestamps and use a sufficiently wide total. */
static int64_t charge_paise(uint64_t energy_wh,
                            int64_t rate_paise_per_kwh,
                            bool peak_period)
{
    int64_t rate = rate_paise_per_kwh;
    /* 20% peak surcharge represented as a rational multiplier.
       This example assumes the resulting paise is rounded down.
       The production policy must define the required rounding rule. */
    if (peak_period) {
        rate = (rate * 120) / 100;
    }
    /* Exact fractional paise may occur; define rounding consistently.
       This simple integer expression truncates, so replace it with the
       co-operative's documented rounding policy if required. */
    return (int64_t)(energy_wh * (uint64_t)rate / 1000u);
}
```

This helper is an illustration, not a complete production smart-meter system. Before submission, explain the rounding rule for fractional paise, validate the multiplication range, and show how timestamps determine whether an interval falls within 18:00–22:00. For intervals crossing 6 PM or 10 PM, split the interval energy proportionally only if the measurement model supports that calculation; otherwise document the meter's interval alignment assumptions.